What is this?

This notebook is part of a collection of `pluto` notebooks on various topics discussed during the Time Domain Astrophysics course delivered by Stefano Covino at the Università dell'Insubria in Como (Italy). Please direct questions and suggestions to stefano.covino@inaf.it.

Table of Contents

MotionSense: Applying SSA to Accelerometer Data

- In this lecture, we'll leave the toy time series behind and apply SSA to real-world data. To this end, we have chosen the accelerometer time series recordings in the MotionSense dataset. More specifically, we will use accelerometer readings taken from a smart phone of a study participant walking naturally.

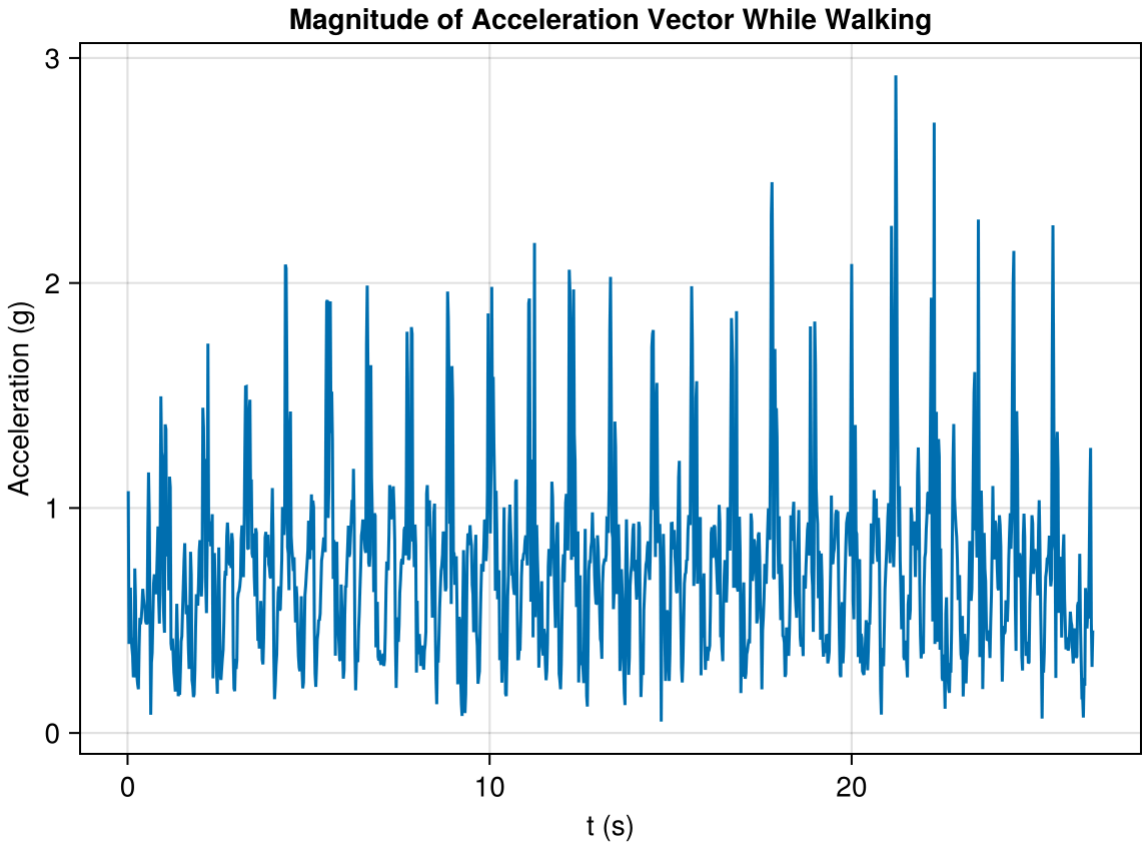
Loading the Data

- The walking time series for each subject are contained in the `sub_*.csv` files. We'll load just Subject 1 for now:

	Column1	attitude.roll	attitude.pitch	attitude.yaw	gravity.x	gravity.y	gravity.z	rotation
1	0	-0.642341	-1.40841	-1.72579	-0.096855	0.986844	-0.129452	-4.8222
2	1	-1.0638	-1.44001	-2.20024	-0.114011	0.991459	-0.063324	-2.6575
3	2	-1.25838	-1.43287	-2.4241	-0.130829	0.990504	-0.042257	-0.6406
4	3	-1.39165	-1.42013	-2.5844	-0.147692	0.988672	-0.026745	-1.3722
5	4	-1.60221	-1.41224	-2.81215	-0.157814	0.987457	0.004959	-1.7648

- The `userAcceleration.*` fields give the components of the instantaneous acceleration vector with respect to the fixed x, y and z coordinate axes, recorded by the participant's smart phone.
- For our purposes, we'll calculate the Euclidean norm of this vector—that is, we'll use the magnitude of the acceleration for our time series.
    - The sampling rate for the measurements is 50 Hz, so we'll also transform the index to units of seconds:

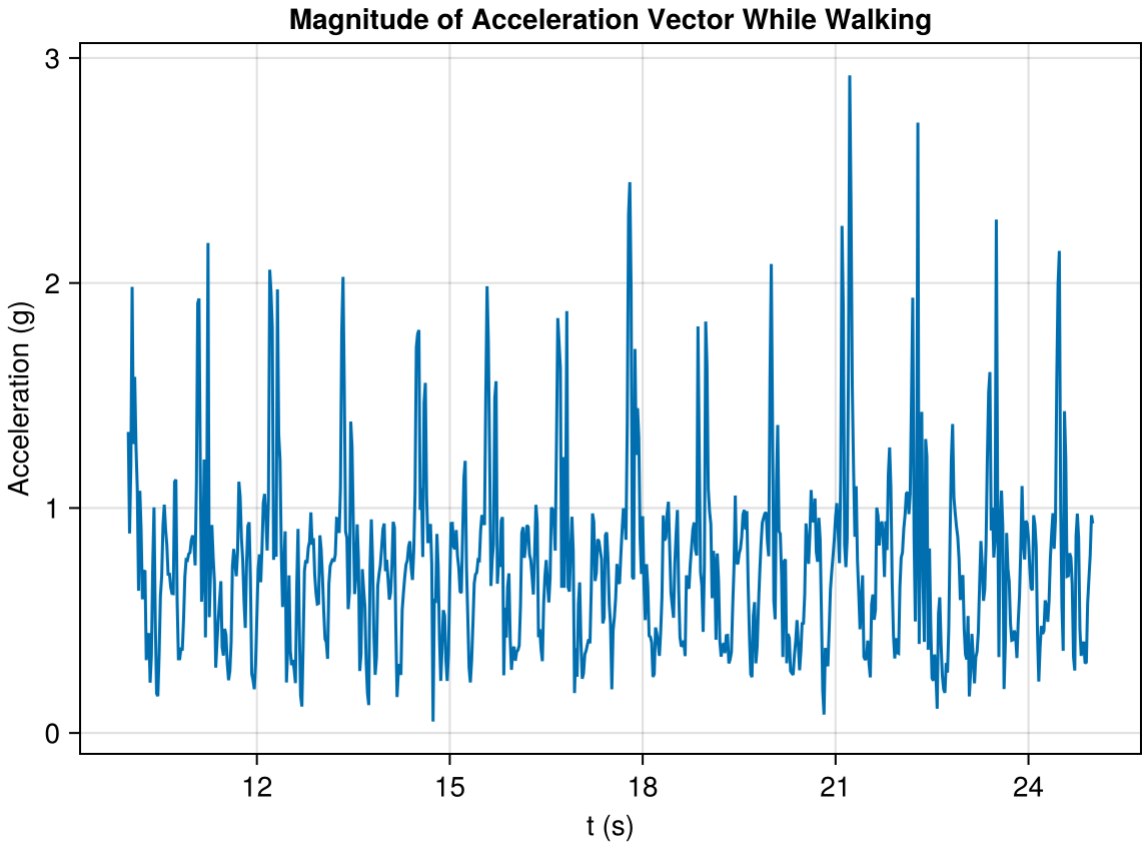
- Let's plot the time series for the magnitude of the acceleration:



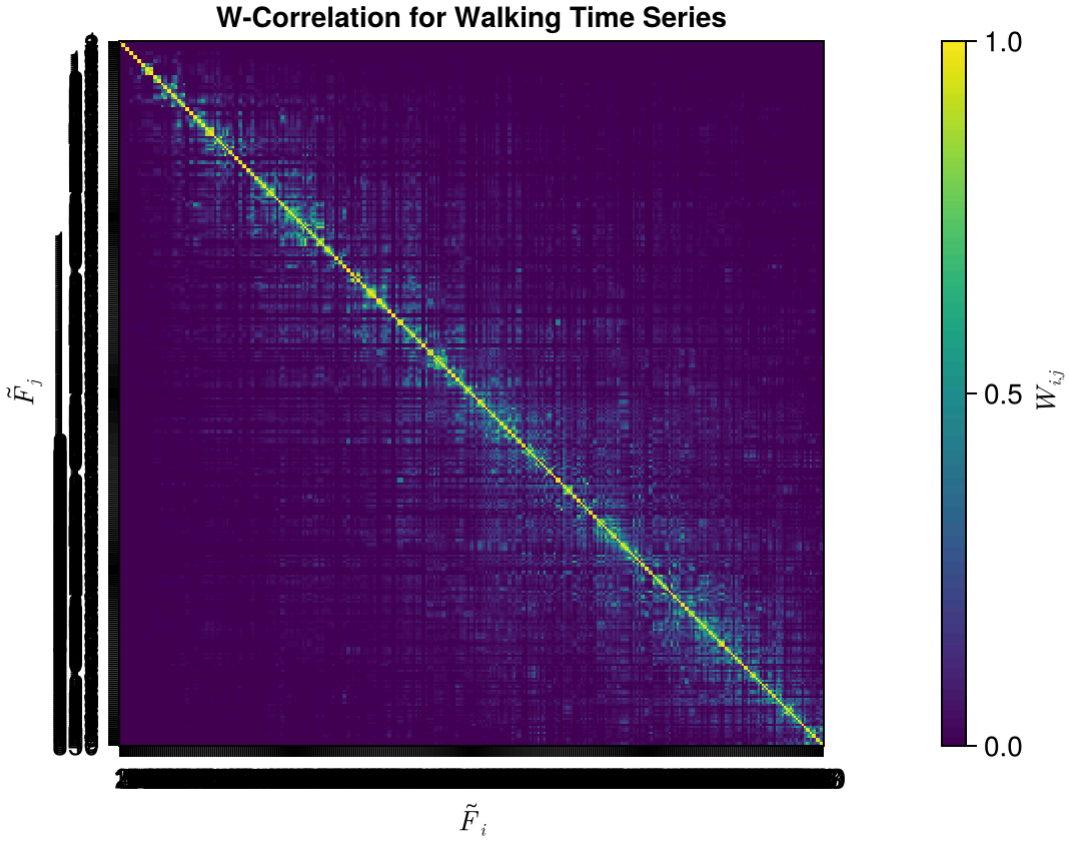
- The instantaneous acceleration over time while walking shows a distinct periodic behavior, as expected.
- We won't attempt to link the regular patterns with the features of walking (i.e. foot-fall, foot-lift, leg-swing, etc.), rather we'll plot a few one-second cycles on top of each other to demonstrate the periodicity in the series:

Decomposing the Time Series With SSA

- As a demonstration, we'll pick a 15-second (750 samples) subseries from the acceleration time series, set a window length of 7 seconds (350 samples), and apply SSA:

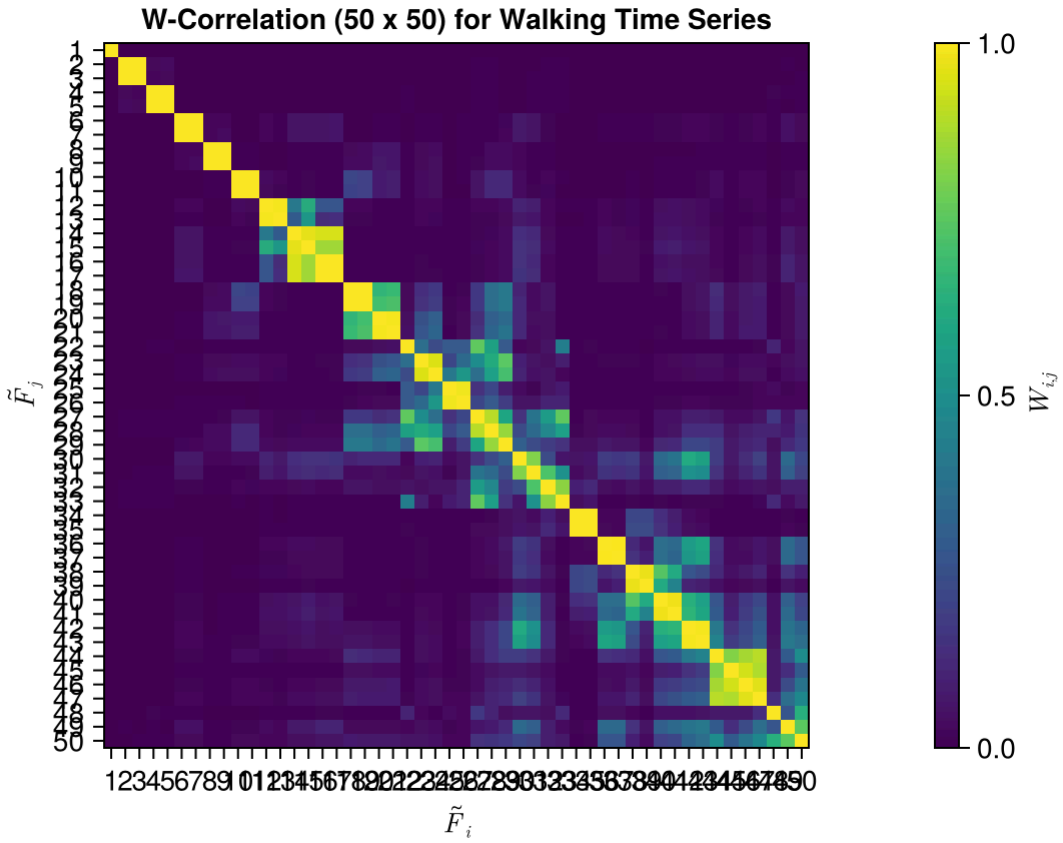


- As always, plot the w-correlation matrix first to help orient ourselves with the separated components.

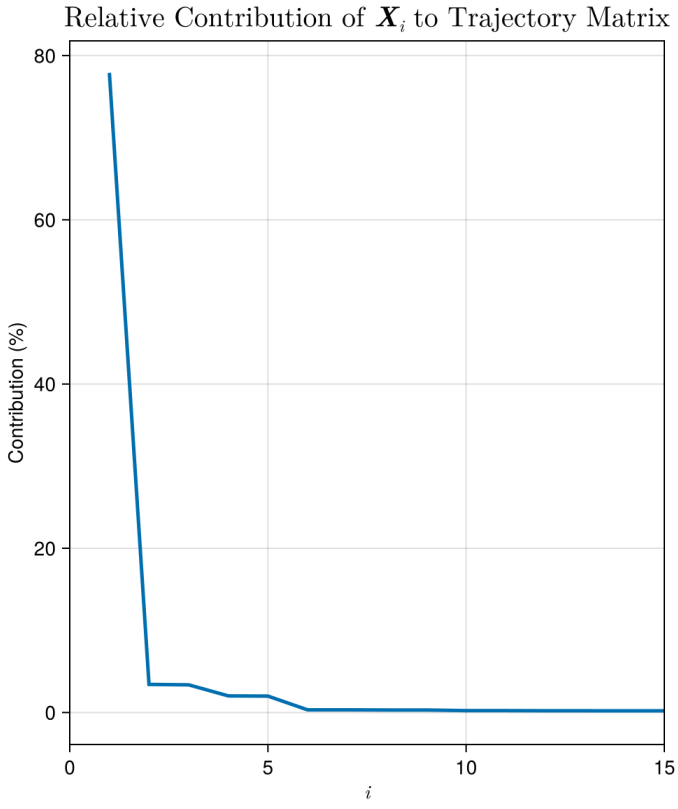


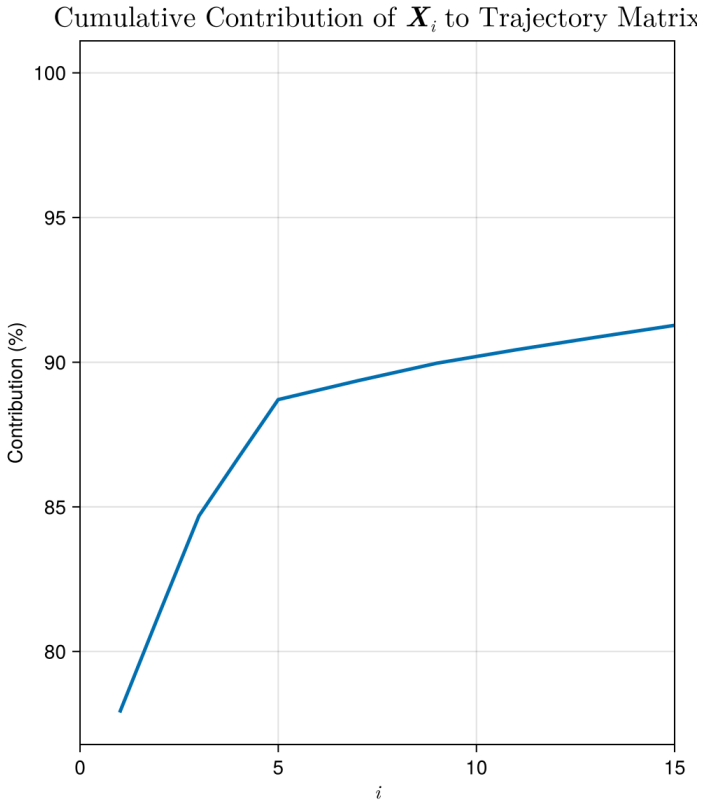
- Of course, with a larger window length (and therefore a large number of elementary components), such a view of the w-correlation matrix is not the most helpful.
- Zoom into the w-correlation matrix for the first components:

Maximum number of components:



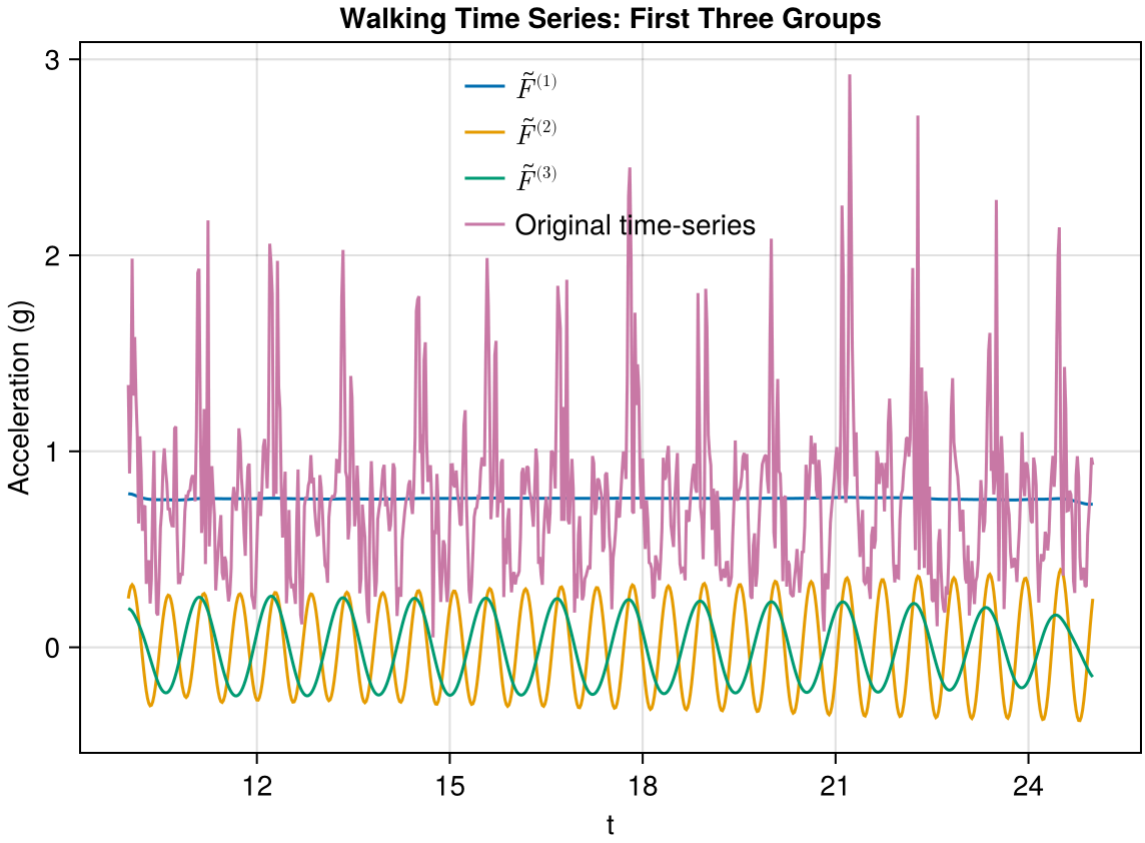
- We can also plot the relative contribution of each component:



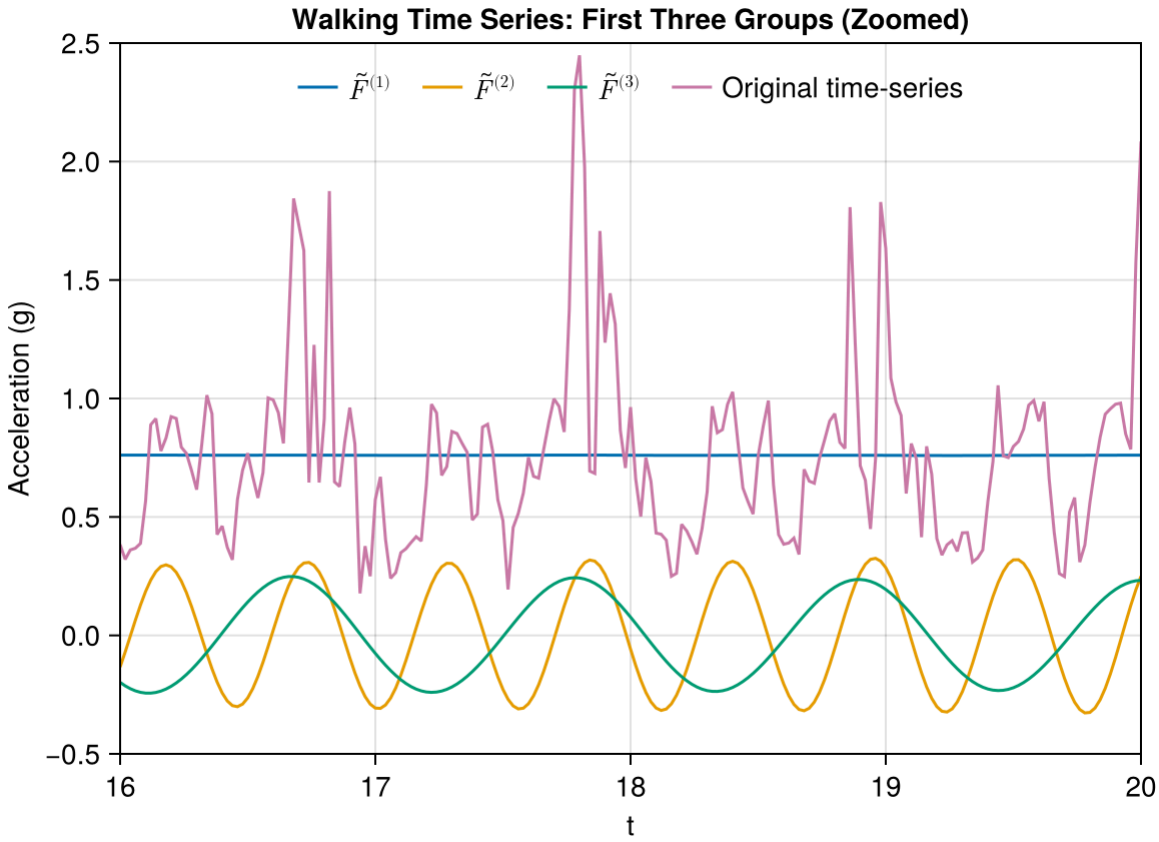


- Beginning with the first five elementary components, we'll form the following groups and plot them:

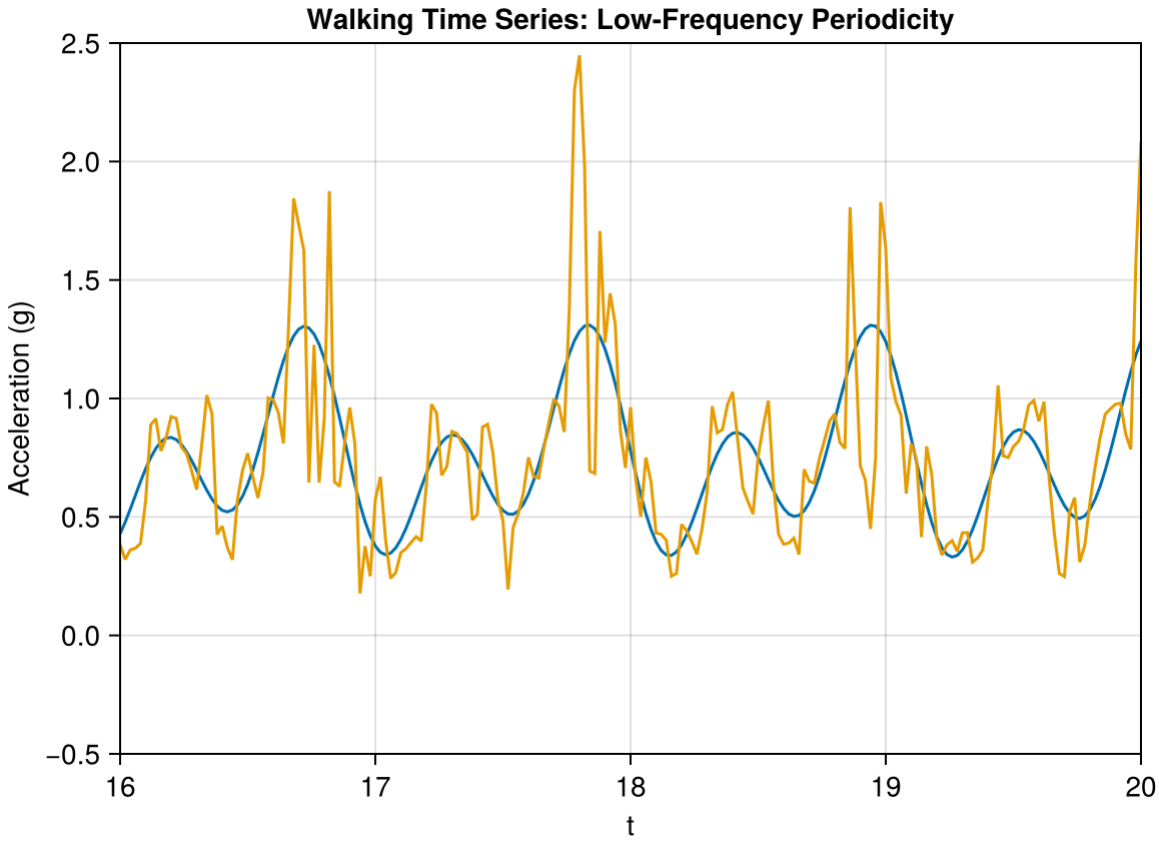
$$\tilde{F}^{(1)} = \tilde{F}_1$$
$$\tilde{F}^{(2)} = \tilde{F}_2 + \tilde{F}_3$$
$$\tilde{F}^{(3)} = \tilde{F}_4 + \tilde{F}_5$$



- And zoom in a four-second subseries:

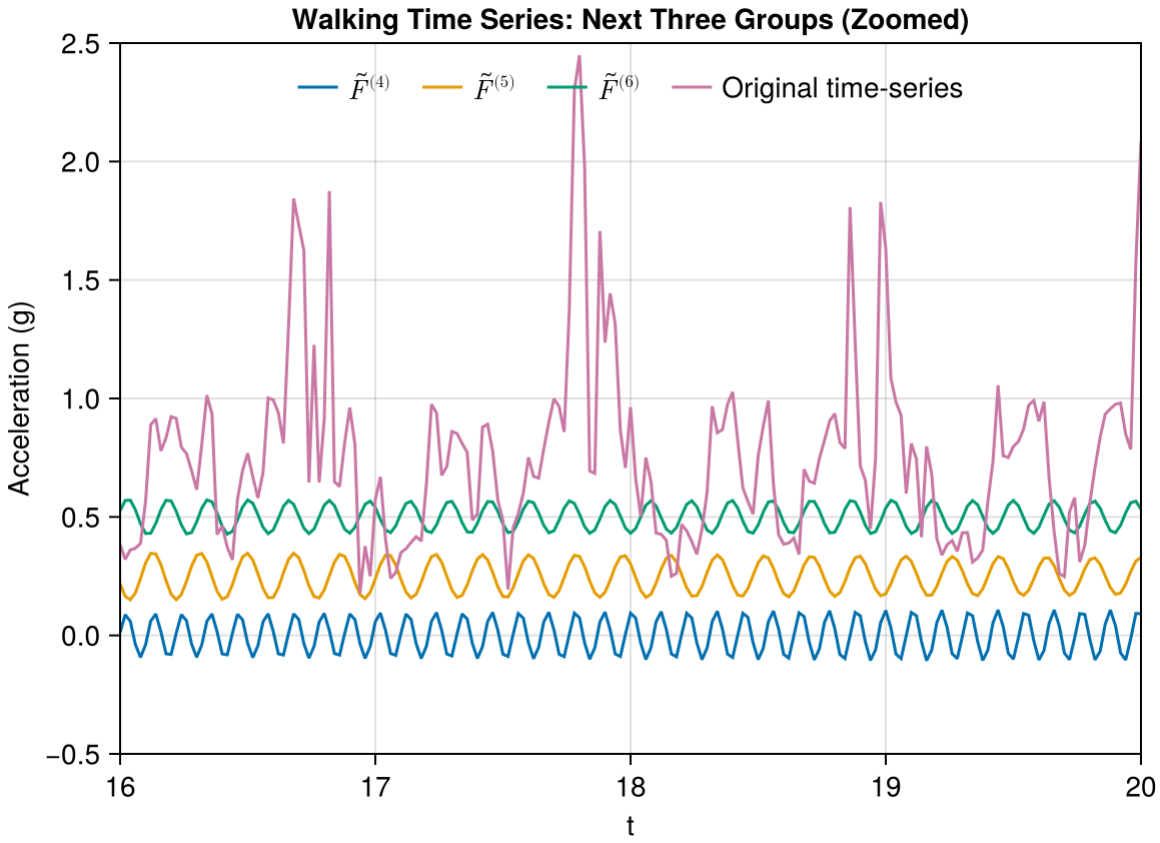


- It is clear from the two plots that the component $\tilde{F}^{(1)}$ is the trend, while $\tilde{F}^{(2)}$ and $\tilde{F}^{(3)}$ are two dominating periodicities.
- As the long-term trend is essentially flat, the subsequent components consist of deviations above and below zero.
- If we decide to add together the first five elementary components, we get a picture of the smooth, low-frequency periodicity present in the series:

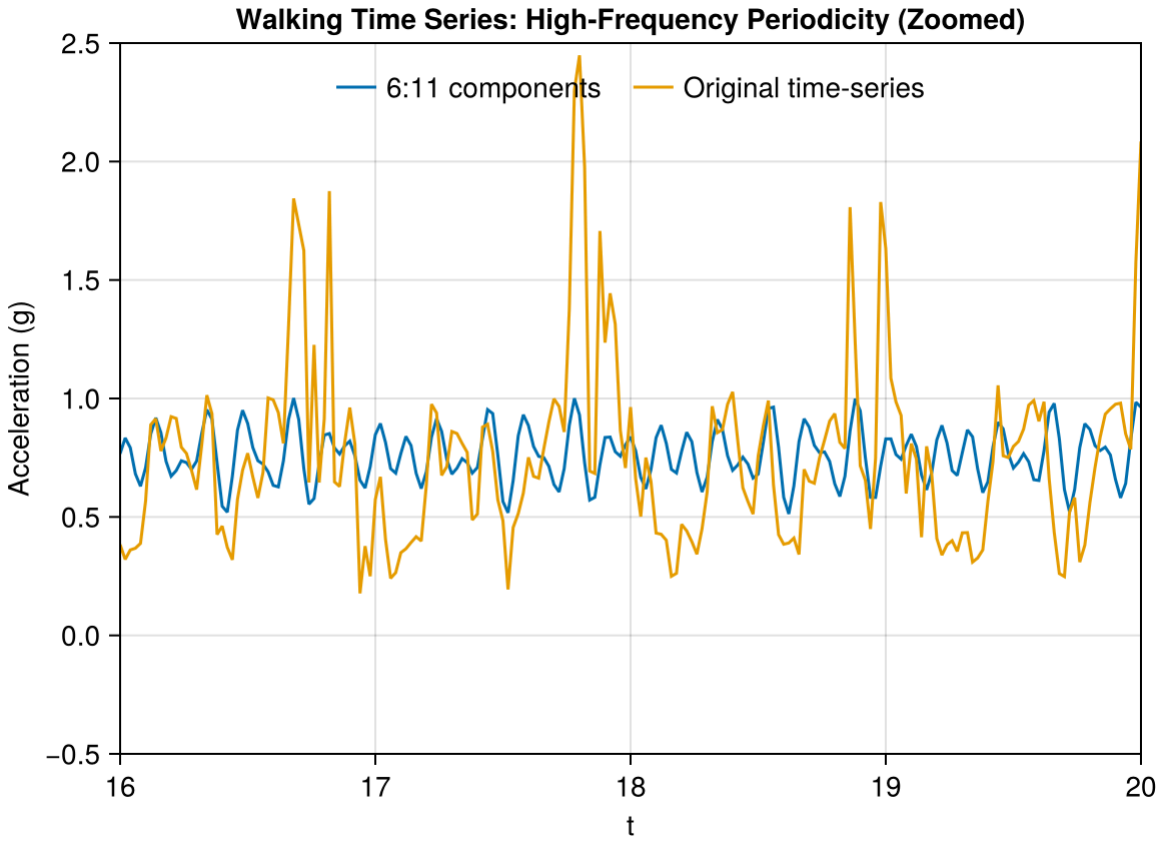


- Closer inspection of the original time series indicates that there are higher frequency periodicities present.
- Let's turn our attention to the elementary components $\tilde{F}_6$ to $\tilde{F}_{11}$. While these components all have some w-correlation with higher i components, we'll make the following grouping:

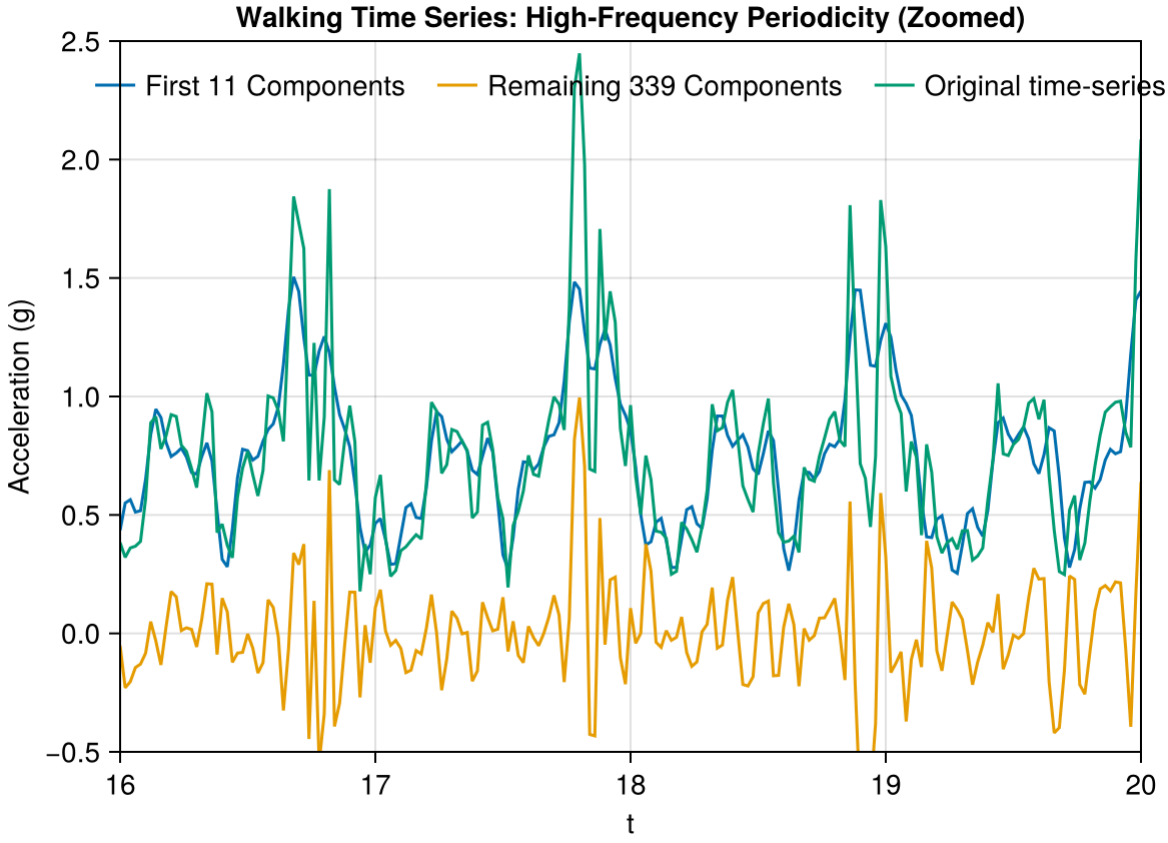
$$\tilde{F}^{(4)} = \tilde{F}_6 + \tilde{F}_7$$
$$\tilde{F}^{(5)} = \tilde{F}_8 + \tilde{F}_9$$
$$\tilde{F}^{(6)} = \tilde{F}_{10} + \tilde{F}_{11}$$



- The grouped components $\tilde{F}_4$, $\tilde{F}_5$ and $\tilde{F}_6$ are three further periodicities, with differing frequencies but approximately the same amplitudes.
- In time series data with a lot of high-frequency components, SSA will typically generate a number of stable harmonic components that are well-separated from each other (i.e. low w-correlation).
    - However, without some domain knowledge of the process generating the time series itself, it is difficult to say whether these components correspond to interpretable processes that operate independently, or if the set of components should be summed and treated as a single component.
- In this case, we'll sum these components together and inspect the periodicity they represent:



- Combining the first 11 elementary components together, and plotting the remaining 339:



- The plot above demonstrates that the sum of the first 11 components adequately capture the basic, underlying periodicity of the accelerometer time series.
- However, the sum of the remaining 339 components (equivalent to the residual series $F_{\text{orig}} - \sum_{i=1}^{11} \tilde{F}_i$) still contains large, regular spikes spaced approximately 1 second apart, likely associated with the footfall events of walking.
- However, the basic form of SSA presented here does not handle these types of sharp periodicities very well, instead spreading the 'signal' from these periodicities across many elementary components.
    - Put another way, the regular part of signal cannot (always, easily) be separated from the noise.

Using SSA to Extract an Individual's Walking 'Signature'

- Let's pretend we were given a dataset consisting of unlabelled walking accelerometer time series, some of them from the same person, perhaps, and tasked with determining the number of individuals in the dataset.
- Let us suppose that everyone's walk has a unique 'signature' that can be extracted from accelerometer data. This signature could be compared across time series to determine whether the time series originates from the same person.
- Given the noise and complexity of the accelerometer time series, we therefore seek a way of extracting a simple, robust signature from the time series data.
- The accelerometer time series we've seen so far contains a simple, regular low frequency component that can easily be extracted by SSA. The walking time series in the MotionSense dataset includes 24 different participants, walking on two separate occasions. We'll attempt to extract the 'signature' component from a handful of these series, and assess if it could act as an identifier for an individual.

This application of SSA has been inspired by the papers Lamar-Leon et al. (2017) and, of course, the source of this dataset is Malekzadeh et al. (2018).

Data Extraction and Processing

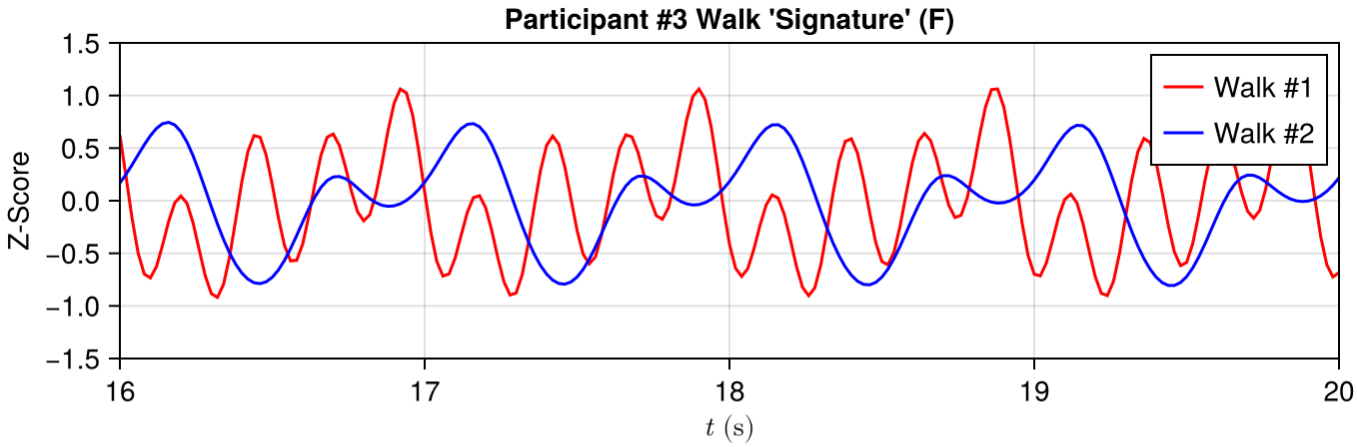
- Below is a subroutine that will extract and return the accelerometer time series for a given subject. The parameter, `walk (= 1 or 2)`, specifies which of the first or second walking time series should be loaded. The time series values are converted to z-scores.
- SSA is then performed on the time series, using the same 15 second subseries and 7 second window length as before. The `SSA` object is then returned.

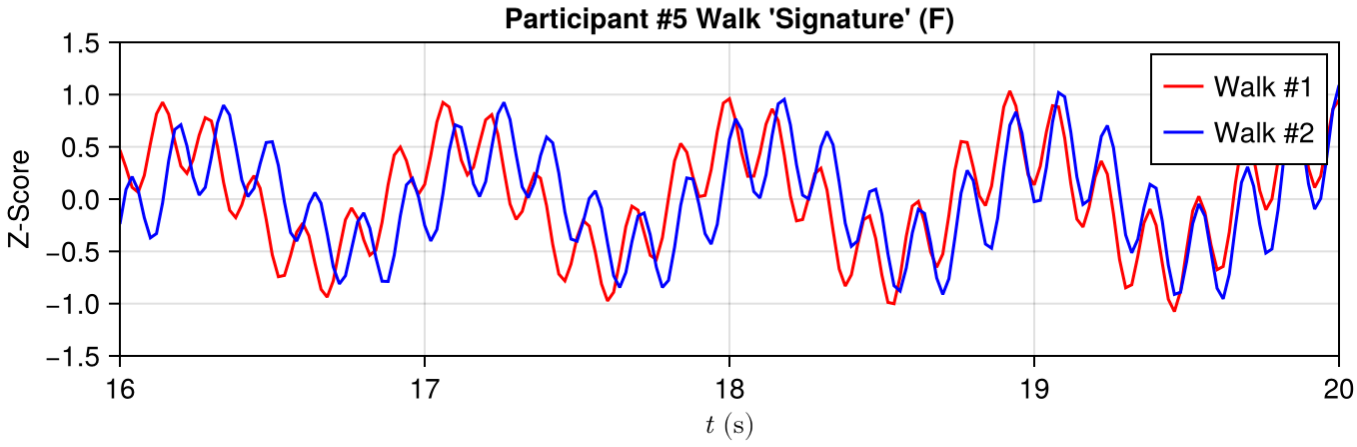
```julia
function extract_series(subject, walk)
    # Mapping walk integers to folder names
    walk_dict = Dict(1 => "wlk_8", 2 => "wlk_15")

    # Constructing the file path
    file_path = "A_DeviceMotion_data/$(walk_dict[walk])/sub_$(subject).csv"

    # Read the CSV into a DataFrame
    df_walk = CSV.read(file_path, DataFrame)

    # Select columns and calculate the magnitude (L2 norm)
    # In Julia, . denotes vectorized operations
    cols = ["userAcceleration.x", "userAcceleration.y", "userAcceleration.z"]
    accel = sqrt.(sum.(eachrow(df_walk[:, cols] .^ 2)))

    # Convert acceleration values to z-scores
    accel = (accel .- mean(accel)) ./ std(accel)

    # Create a time index (assuming 50Hz based on your Python code)
    # Julia's index is 1-based, so we create a range for the time axis
    time_index = (0:length(accel)-1) ./ 50

    # Filter by time (10 to 25 seconds)
    start_time, end_time = 10.0, 25.0
    mask = (time_index .>= start_time) .& (time_index .<= end_time)
    accel_subset = accel[mask]

    # Perform SSA (Assuming you have an SSA function defined elsewhere)
    window = 350
    return SSAJL.SSA(accel_subset, window)
end;
```

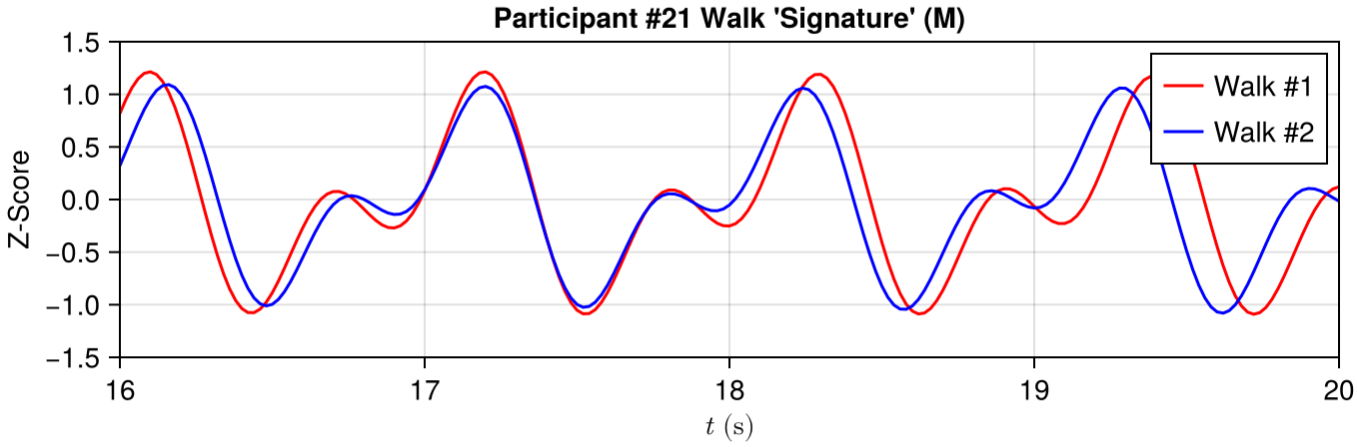
Comparison of Walk 'Signatures'

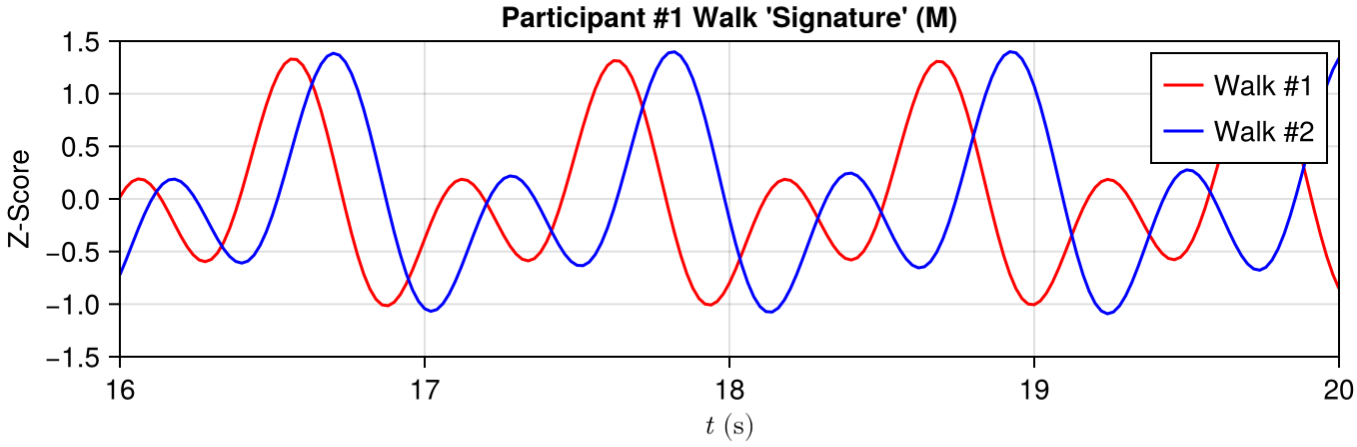
- For this task, we'll only do an informal comparison of the walks from several participants. A more rigorous investigation would properly quantify time series similarity, ensure that the different time series are aligned as best as possible before comparison, and account for differences in walking pace.
- Here, we'll plot a few time series and compare them by eye. We will assume the low frequency components of interest are all contained in the first four eigentriples.
- Based on the provided participants' characteristics here, the participants we'll examine are as follows:
    - `3` and `5`—both females with almost-identical weights and heights;
    - `21`—male with similar age, height and weight to participants `3` and `5`; and
    - `1` and `22`—two males with similar heights and weights, approximately twice as heavy as participants `3`, `5` and `21`.

- Plot all of the extracted 'signatures', using the first four eigentriples of the SSA decomposition:


Participant #3 Walk 'Signature' (F)
Z-Score
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
16
17
18
19
20
t (s)
Walk #1
Walk #2


Participant #5 Walk 'Signature' (F)
Z-Score
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
16
17
18
19
20
t (s)
Walk #1
Walk #2


Participant #21 Walk 'Signature' (M)
Z-Score
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
16
17
18
19
20
t (s)
Walk #1
Walk #2


Participant #1 Walk 'Signature' (M)
Z-Score
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
16
17
18
19
20
t (s)
Walk #1
Walk #2


Participant #22 Walk 'Signature' (M)
Z-Score
1.5
1.0
0.5
0.0
−0.5
−1.0
Walk #1
Walk #2

- With the exception of Participant 3, the 'signature' extracted from the participants' walks are almost identical between Walk 1 and Walk 2, their relative offsets notwithstanding. While the overall time series for the male participants are very similar, we could derive some form of similarity measure to discriminate between them. For interested readers, Serrà and Arcos (2014) provide a review of time series similarity measures.
- While this is a very rudimentary approach to discriminating between identities in time series data, it demonstrates how SSA can be easily applied to extract simple features from complicated, noisy time series.

Final Words

- Through a relatively straightforward process of embedding, decomposition and reconstruction, the technique of *singular-spectrum analysis* can extract the trend of a time series, separate underlying periodicities and remove noise. It can be used as an exploratory tool, or in the context of a more detailed analysis.
- The version of singular-spectrum analysis presented here is typically termed *basic SSA*, as a number of variants and extensions to the method have been developed. Singular-spectrum analysis can also be used for forecasting, and detecting structural changes in a time series, that is, detecting where a time series has been perturbed and taken on a new 'behaviour'. Multivariate and 2D versions of singular-spectrum analysis also exist.

Reference & Material

Material and papers related to the topics discussed in this lecture.

- Lamar-Leon et al. (2017) - "Persistent-homology-based gait recognition"
- Malekzadeh et al. (2018) - "Protecting Sensory Data against Sensitive Inferences"

Credits

This notebook is obtained and further elaborated from: https://www.kaggle.com/code/jdarcy/introducing-ssa-for-time-series-decomposition with data from https://github.com/mmalekzadeh/motion-sense.

Course Flow

	Previous lecture	Next lecture	Course Summary
notebook	Lecture about singular spectrum analysis	Lecture about Gaussian processes	Course Summary
html	Lecture about singular spectrum analysis	Lecture about Gaussian processes	Course Summary

Notebook v1.0.0 - 09 May 2026